

ResNet18 implementation and analysis on CIFAR-10

Andrea Colombo - Artificial Intelligence course project - A.A 2025/2026

Abstract — This report presents a comprehensive implementation and analysis of a ResNet18, a deep residual network architecture for image classification. We implement the network from scratch using PyTorch, train it on the CIFAR-10 dataset and analyze its performance characteristics. This implementation achieves competitive accuracy, showcasing the effectiveness of residual network in enabling deep network training.

Table of Contents

1	Introduction	3
1.1	Motivation and Objectives	3
2	Background	4
2.1	Residual Learning Framework	4
2.2	Related Work	4
3	Dataset Description	4
3.1	Class Distribution	5
3.2	Dataset Split	5
3.3	Dataset Characteristics	5
3.4	State Of The Art	6
4	Implementation Details	6
4.1	Development environment	6
4.2	Network architecture	6
4.2.1	Overall architecture	6
4.2.2	Residual Block Implementation	7
4.2.3	Full Architecture Implementation	8
5	Training Methodology	8
5.1	Data Augmentation	8
5.2	Early Stopping	9
5.3	Defining a Validation Set	9
5.4	Optimization Strategy	9
5.5	Loss Function	9
6	Experimental Results	10
6.1	Training Configuration	10
6.2	Training performance	10
6.3	Results on Test Dataset	10
6.4	Results Analysis	11
6.4.1	Per Class Performance	11
7	Conclusions	12
7.1	Key Takeaways	12
7.2	Limitations	12
7.3	Possible Improvements	13

Bibliography	13
--------------------	----

1 Introduction

Deep convolutional neural networks have revolutionized computer vision, but training very deep networks has always been extremely challenging due to the degradation problem: as the network’s depth increases, accuracy saturates and then rapidly degrades. Critically, this degradation is not caused by overfitting but, instead, by optimization difficulties that prevent deeper networks from learning even simple identity mappings.

Residual networks (ResNets) were introduced in [1] by He et al in 2015. These networks address this fundamental issue through an extremely elegant architectural innovation called *skip connections*. A skip connection allows information to bypass layers via an identity mapping. Instead of forcing each layer to directly learn an underlying mapping $\mathcal{H}(\mathbf{x})$, ResNets reformulate the problem to learn a residual function $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$.

This architectural change has very deep implications; if the optimal function is close to an identity mapping, it’s far easier to push the residual term towards zero than to fit an identity mapping through multiple non-linear functions. This allows the training of networks that are much deeper than the ones used before this architecture was invented.

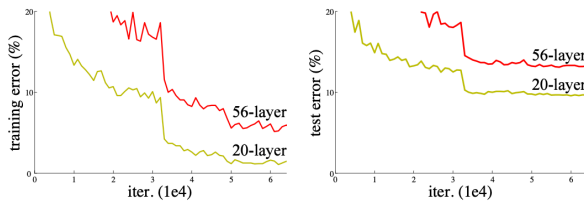


Figure 1: Visualization of the degradation problem through a graph of training errors at different depths, image from [1]

1.1 Motivation and Objectives

This project implements a particular residual network architecture called ResNet18 and evaluates it on CIFAR-10, a widely used benchmark for image classification.

Our objectives are the following:

- Implement a ResNet18 from scratch using PyTorch and adapt it to the CIFAR-10 dataset.
- Train the model to achieve competitive accuracy
- Analyze the role of residual connections

The remainder of this document is organized as follows:

- Section 2 provides background on residual learning.
- Section 3 provides information on the used dataset and related challenges.
- Section 4 contains the implementation details.
- Section 5 presents the training methodology.
- Section 6 analyzes experimental results.
- Section 7 contains the final thoughts about the project

2 Background

2.1 Residual Learning Framework

The fundamental innovation in residual networks is the residual block.

Traditional neural networks learn a *direct* mapping from input to output. ResNets, instead, learn the residual (difference) between input and output.

Formally, given an input $\mathbf{x}$, a traditional neural network learns the following:

$$\mathbf{y} = \mathcal{H}(\mathbf{x})$$

A residual network instead learns:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$$

where $\mathcal{F}(\mathbf{x}, \{W_i\})$ represents the residual mapping learned by a number of stacked layers, and the identity mapping $\mathbf{x}$ is added via the skip connection.

This formulation is based on the hypothesis that learning the residual mapping $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$ is easier than learning a complete mapping $\mathcal{H}(\mathbf{x})$. Additionally, if the optimal mapping is close to the identity mapping, the network can simply drive the residual term towards zero, which is simpler than fitting the identity mapping through multiple non-linear layers.

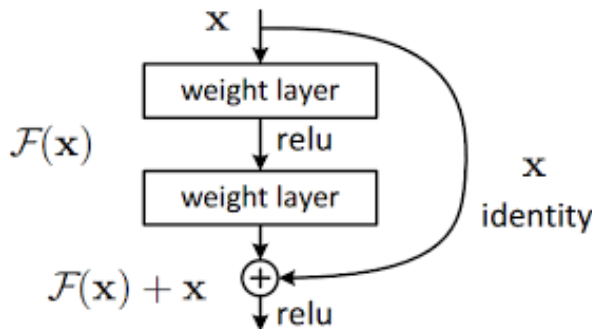


Figure 2: Residual block visual representation

2.2 Related Work

The evolution towards residual networks followed several important developments in deep learning:

- **Early CNNs:** LeNet and AlexNet demonstrated the power of convolutional architectures but were limited to shallow networks.
- **VGG Networks:** Showed that performance could be improved by using deeper networks but training became increasingly difficult.
- **The Degradation problem:** it was observed that simply adding more layers led to a higher training error, indicating that it was not caused by overfitting but rather by optimization difficulties.

3 Dataset Description

The CIFAR-10 dataset [2] is one of the most widely adopted benchmark datasets for evaluating image classification algorithms in computer vision research. It serves as a testbed to compare

different model architectures, training methods and optimization techniques.

The dataset consists of 60,000 images of size 32×32 pixels, each one of them belonging to one of ten mutually exclusive classes.

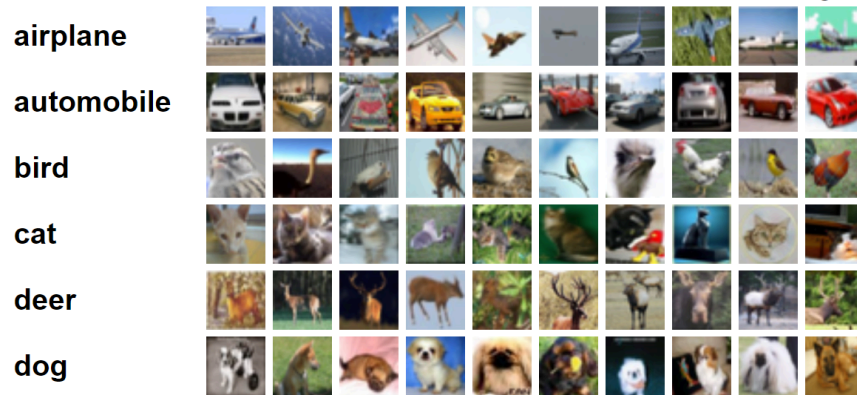


Figure 3: Example of images from the CIFAR-10 dataset

3.1 Class Distribution

The dataset is organized into 10 classes that represent common objects or animals.

The classes are equally present inside the dataset, with each class containing 6,000 images (5,000 for the training set and 1,000 for the test set).

3.2 Dataset Split

The dataset is pre-divided into training and test sets in the following way:

- **Training set:** 50,000 images (5,000 per class)
- **Test set:** 10,000 images (1,000 per class)

For our purpose, the training set was further divided to create a validation set for hyperparameter tuning and early stopping, leaving us with the following split:

- **Training subset:** 45,000 images
- **Validation subset:** 5,000 images
- **Test set:** 10,000 images (unchanged)

3.3 Dataset Characteristics

Despite its relatively small size, the CIFAR-10 dataset poses several challenges that make it a meaningful benchmark:

- **Low resolution:** The 32×32 pixel size requires the model to learn meaningful high level features
- **Intra-class variability:** The objects and animals represented in the pictures appear in various different poses, with different lighting and backgrounds, this means that the model will need to have strong generalization capabilities.
- **Inter-class similarity:** Some classes share visual similarities between them, making it difficult to distinguish between them.

3.4 State Of The Art

Modern architectures are capable of achieving test accuracies exceeding 99% on the CIFAR-10 dataset using advanced techniques such as vision transformers, extensive data augmentation and semi-supervised learning. Standard ResNet architectures generally achieve 92-95% accuracy. The goal of this project is to match or possibly exceed this baseline.

4 Implementation Details

4.1 Development environment

This project was implemented in PyTorch, a leading deep learning framework that provides great tooling for building neural networks.

Some of its advantages are:

- **Dynamic computation graphs** enable easier debugging and experimentation.
- **Strong community support** and extensive documentation. At the time of writing, PyTorch is the most used deep learning frameworks amongst researchers.
- **Efficient GPU acceleration** through CUDA integration, making it a great option for working with Google CoLab.
- **Built-in dataset support** via torchvision.

4.2 Network architecture

The ResNet18 architecture consists of an initial convolutional layer followed by four stages, each composed of two residual blocks. Each stage increases the number of channels while reducing spatial dimensions, allowing the network to learn increasingly abstract features in later layers.

4.2.1 Overall architecture

The complete network architecture described above is reported in the following table.

Layer Name	Output Size	Configuration
conv1 + bn1	$32 \times 32 \times 64$	3×3 , 64, stride=1
res_1	$32 \times 32 \times 64$	$[3 \times 3, 64] \times 2 \times 2$, stride=1
res_2	$16 \times 16 \times 128$	$[3 \times 3, 128] \times 2 \times 2$, stride=2
res_3	$8 \times 8 \times 256$	$[3 \times 3, 256] \times 2 \times 2$, stride=2
res_4	$4 \times 4 \times 512$	$[3 \times 3, 512] \times 2 \times 2$, stride=2
avg_pool	$1 \times 1 \times 512$	4×4 average pooling
linear	10	$512 \rightarrow 10$

Table 1: ResNet18 architecture for CIFAR-10. Each res_X stage contains 2 residual blocks. First block in stages 2-4 uses stride=2 for spatial downsampling.

4.2.2 Residual Block Implementation

The residual block is the core of any residual network, the implementation used in this project uses two convolutional layers with a kernel size of 3 followed by batch normalization.

This implementation uses *post activation*, following the architecture proposed in the first paper.

An alternative to this approach, called *pre activation*, was proposed in [3] by He et al.

This technique mainly benefits really deep residual networks so it wasn't used for this particular project. What follows is the python code used to implement a residual block in this project.

```
python

class ResBlock(nn.Module):

    def __init__(self, in_channels, out_channels, stride=1):
        super(ResBlock, self).__init__()
        self.conv_1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,
                                stride=stride, padding=1, bias=False)
        self.bnorm_1 = nn.BatchNorm2d(out_channels)

        self.conv_2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
                                stride=1, padding=1, bias=False)
        self.bnorm_2 = nn.BatchNorm2d(out_channels)
        # Each residual block has a skip connection, this is what separates
        # residual networks from the architectures that were previously used.
        # If the dimensions and stride allow it, the skip connection is the
        # identity mapping (used to propagate the block input forward).
        # If the dimensions do not match (or stride != 1) we have to use a
        # projection mapping to match the dimensions
        self.skip_connection = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.skip_connection = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, kernel_size=1,
                          stride=stride, bias=False),
                nn.BatchNorm2d(out_channels)
            )

    def forward(self, x):
        # The idea of a forward pass is to have a  $H(x) = F(x) + x$ , where  $F(x)$ 
        # is the result of the first and second convolutional layers (residual
        # path) and  $x$  is the input that gets propagated via the skip connection.
        out = F.relu(self.bnorm_1(self.conv_1(x)))
        out = self.bnorm_2(self.conv_2(out))
        out += self.skip_connection(x)
        out = F.relu(out)
        return out
```

Listing 1: Residual block implementation in python using PyTorch

4.2.3 Full Architecture Implementation

The architecture of the full residual network, as described in the previous sections, is implemented using the following python code:

```
python

class ResNet18(nn.Module):

    def __init__(self):
        super(ResNet18, self).__init__()
        self.conv_1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1,
                                bias=False)
        self.bnorm_1 = nn.BatchNorm2d(64)
        # Residual blocks configuration: after the first layer we use a stride
        # of 2 in order to downsample spatially (since we are modifying the
        # number of channels)
        self.res_1 = self.make_blocks(64, 64, 2, stride=1)
        self.res_2 = self.make_blocks(64, 128, 2, stride=2)
        self.res_3 = self.make_blocks(128, 256, 2, stride=2)
        self.res_4 = self.make_blocks(256, 512, 2, stride=2)
        # Final linear layer that maps to the output classes
        self.linear = nn.Linear(512, 10)

    def forward(self, x):
        out = F.relu(self.bnorm_1(self.conv_1(x)))
        out = self.res_1(out)
        out = self.res_2(out)
        out = self.res_3(out)
        out = self.res_4(out)
        out = F.avg_pool2d(out, 4)
        out = out.view(out.size(0), -1)
        out = self.linear(out)
        return out

    def make_blocks(self, in_chs, out_chs, n, stride):
        layers = []
        layers.append(ResBlock(in_chs, out_chs, stride))
        for _ in range(1, n):
            layers.append(ResBlock(out_chs, out_chs, 1))
        return nn.Sequential(*layers)
```

Listing 2: Python code for the ResNet18 architecture using PyTorch

The network defined above contains 11,173,962 trainable parameters.

5 Training Methodology

When training deep neural networks, we have to pay attention to make sure the network does not overfit the dataset. Some of the choices in the training process were made specifically to reduce the likelihood of overfitting.

5.1 Data Augmentation

In order to improve the network’s generalization capabilities, standard data augmentation techniques were applied to the dataset, including random horizontal flipping and random cropping. Data augmentation was **not** applied to the test and validation sets.

5.2 Early Stopping

To prevent overfitting and reduce unnecessary computation, early stopping was applied during the training process.

Early stopping is a regularization technique used to prevent overfitting by stopping the training after the model’s performance on the validation set stopped improving for a certain number of epochs (patience).

Models trained with early stopping tend to have better generalization capabilities.

5.3 Defining a Validation Set

In order to use early stopping, we have to validate the model using some data and, for obvious reasons, we can’t use the training data for this purpose.

As stated in **Section 3.2**, we use a 90-10 split over the training set where the 10% will be used for validation during the training process.

5.4 Optimization Strategy

The presented model was trained using the AdamW optimizer [4], a variant of the Adam optimizer that brings some key advantages over more traditional optimizers like SGD.

- **Adaptive learning rates:** AdamW maintains per-parameter learning rates that adjust based on gradient history. This allows different layers of the network to learn at different speeds without manual tuning.
- **Robust convergence:** AdamW is less sensitive to hyperparameter values, making it a more reliable choice compared to SGD.
- **Better weight decay:** AdamW decouples weight decay from gradient updates. This often leads to better generalization which can be helpful on the CIFAR-10’s small training set.

5.5 Loss Function

We employ Cross-Entropy loss, a standard choice for multi class classification tasks.

Given a training example with true class y and predicted probabilities $\hat{\mathbf{p}}$, the loss is defined as follows:

$$\mathcal{L}_{\text{CE}} = -\log(\hat{p}_y) = -\log\left(\frac{e^{z_y}}{\sum_{j=1}^C e^{z_j}}\right)$$

Where z_j are the logits for each $C = 10$ classes.

Cross-entropy is particularly suited for the CIFAR-10 classification task for the following reasons:

- It measures the dissimilarity between the true distribution (one-hot encoded labels) and the predicted distribution, directly optimizing the model to produce well-calibrated probability estimates.
- When the model is confident but wrong, the loss produces large gradients that drive rapid correction.

6 Experimental Results

6.1 Training Configuration

The model was trained following the methodology described in Section 5 with early stopping based on the validation loss.

As stated in Section 5.1, data augmentation techniques were only applied to the training set.

The following table describes the hyperparameters used for the training process:

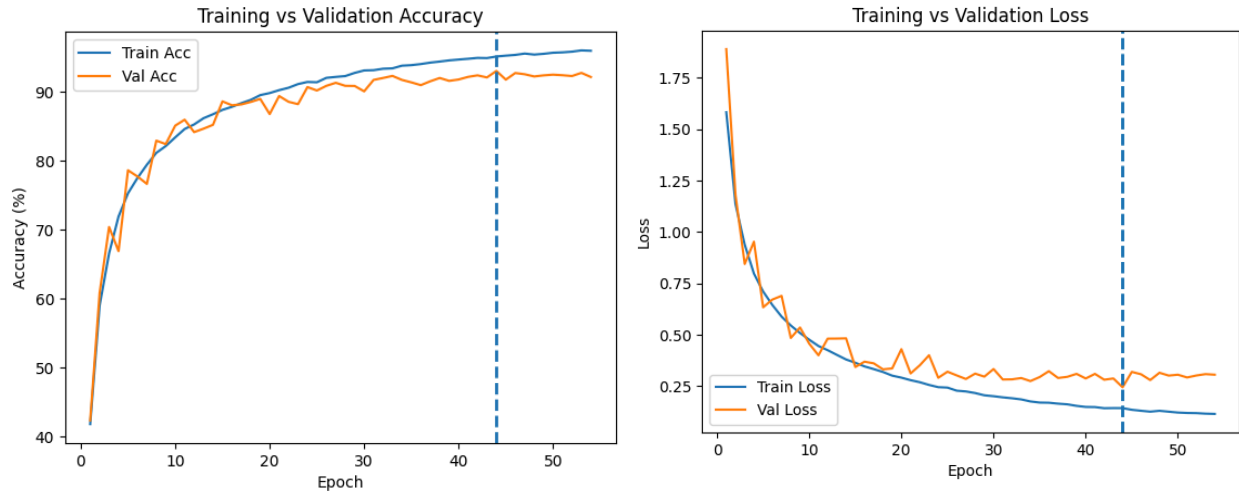
Parameter	Value
max epochs	100
batch size	128
learning rate	1e-3
weight decay	1e-4
patience	10
min delta	1e-3

6.2 Training performance

The training was conducted for 54 epochs before early stopping was triggered. The model showed strong initial learning, reaching approximately 85% accuracy within the first 10 epochs.

After that, the performance continued to improve, with the validation accuracy reaching its peak of 93.04% at epoch 44.

The following plots show the loss and accuracy curves for the training and validation sets.



6.3 Results on Test Dataset

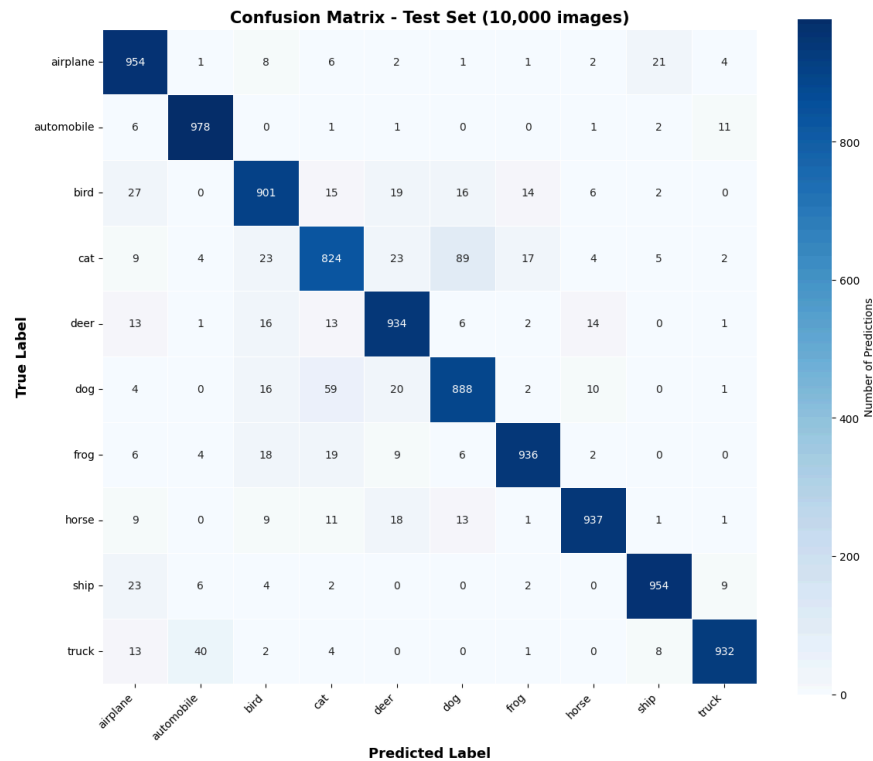
The trained model achieved a final accuracy of 92.38% against the CIFAR-10 test set. This value falls in the standard performance baseline for ResNet18 networks, showing a successful implementation of the architecture.

6.4 Results Analysis

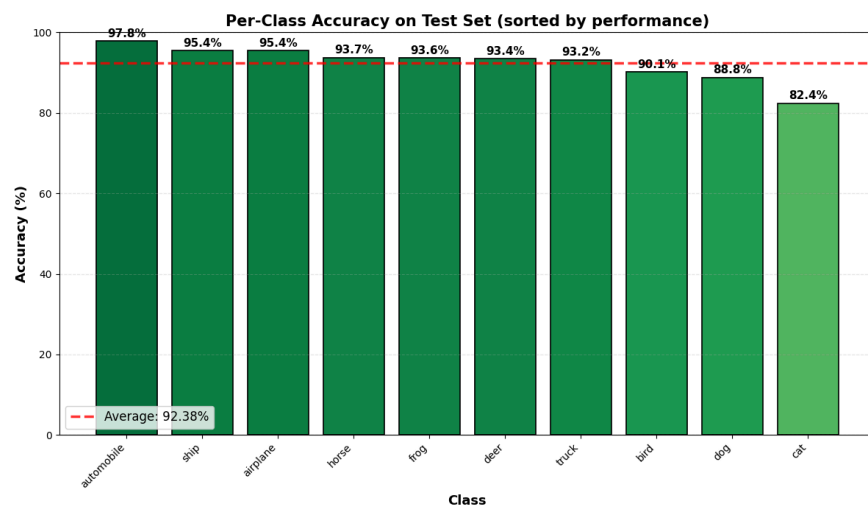
6.4.1 Per Class Performance

To better understand the model behavior, we analyzed its performance on each individual class, using standard analysis tools like the confusion matrix.

Using the confusion matrix helps us understand which classes are the most difficult to classify for our model.



The model achieved the following per-class accuracies:



We can see how the best performing classes are the ones that present distinctive visual features even at such a low resolution. The ones that performed the worst are the ones with a high degree of inter-class similarities. In particular, the most frequent confusion occurs between cats and dogs, this is due to the fact that, at such low resolutions, these classes have similar poses, body shapes and textures that make classification difficult.

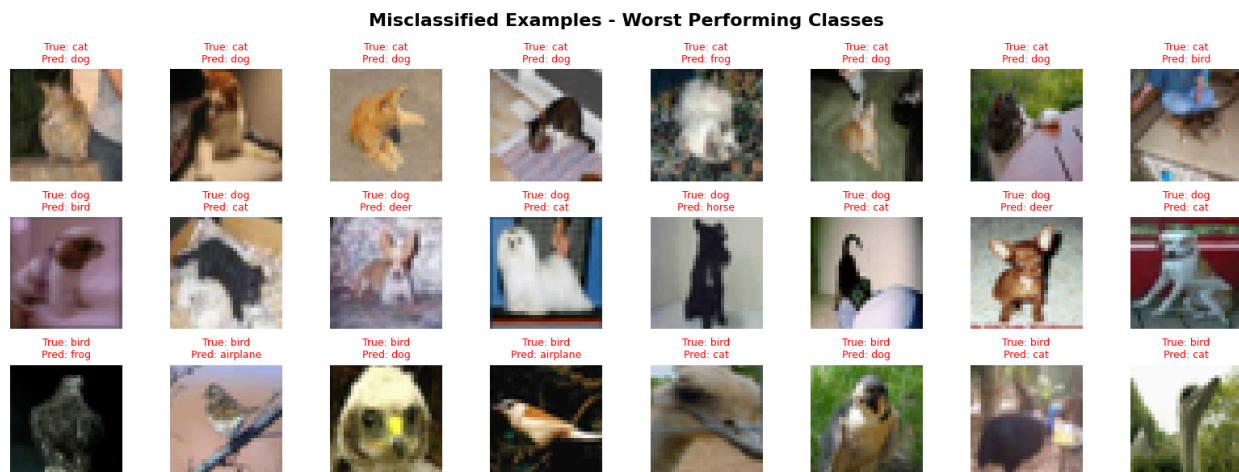


Figure 8: Examples of wrongly classified images in the 3 worst performing classes

Despite visual similarities, the model was able to correctly distinguish between automobiles and trucks with minimal confusion. This shows that the model was able to successfully learn meaningful features.

7 Conclusions

The implementation described in this report achieved 92.38% test accuracy on the CIFAR-10 dataset, meeting the expected baseline for the ResNet18 architecture.

The small gap between validation accuracy (93.04%) and test accuracy (92.38%) shows that the model was able to learn meaningful features instead of simply memorizing data.

7.1 Key Takeaways

- **Residual connections worked:** The skip connections enabled effective training of the deep network by allowing the gradients to flow directly. This architectural choice successfully addressed the degradation problem found in earlier architectures.
- **Regularization was effective:** The combination of data augmentation, early stopping and weight decay prevented severe overfitting.
- **Performance between classes:** as it was expected, the classes with the best accuracy were the ones with distinctive visual features, while the ones with lower accuracy were the ones that presented more inter-class similarities.

7.2 Limitations

Although the implementation met the expected results, it's not without flaws, in this section we explore some of the details of this model that could be improved.

- **Minor Overfitting:** The gap between training accuracy and test accuracy of the saved model of 2.76% indicates that the model memorized some patterns of the training data.

Although the gap is relatively small and acceptable, stronger regularization techniques, more advanced data augmentation and learning rate scheduling could possibly reduce this gap and improve the final performance of the model.

- **Single dataset performance:** The model’s architecture was trained only on the CIFAR-10 dataset, therefore performance on other datasets and transfer learning capabilities were not explored.

7.3 Possible Improvements

The analysis made on the model’s performance showed some promising directions that could improve upon this work.

- **Advanced augmentation:** more modern data augmentation techniques such as *AutoAugment* could potentially improve the generalization capabilities of the model and result in better accuracy on test data.
- **Ablation studies:** Removing components and measuring the impact would give us a better understanding on which architectural choices matter more.
- **Transfer learning experiments:** Pre-training the model on other similar datasets (e.g. ImageNet) and then fine tuning the model on CIFAR-10 would enable us to assess the performance tradeoffs and whether or not transfer learning is a viable option.

Bibliography

- [1] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- [2] A. Krizhevsky and G. Hinton, “Learning Multiple Layers of Features from Tiny Images,” technical report, 2009.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, “Identity Mappings in Deep Residual Networks,” in *Proceedings of the European Conference on Computer Vision*, 2016.
- [4] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proceedings of the International Conference on Learning Representations*, 2019.